

Introduction to Buffer

- **Buffer:** A temporary storage area used to hold data while it is being transferred between two locations (like memory ↔ disk, or memory ↔ network).
- **Purpose:** To handle **data speed mismatch** between sender and receiver.
- Common in **streaming, file I/O, networking**.
- Reduces the number of **read/write operations**.

📌 **Think of it like a waiting room for data.**

Why use Buffer?

- Node.js is good with I/O operations — buffers help in handling data chunks before processing.
- Especially useful when data is not in UTF-8 (like images, videos, etc.)

ArrayBuffer

- **ArrayBuffer:** Array of bytes **or** A low-level object used to represent a **fixed-length binary data** buffer.
- It stores raw bytes and is not directly readable — needs a **view** like `TypedArray` or `DataView`.
- Common in **binary file processing, Web APIs**, and **streams**.
- Immutable size – once created, you can't resize it.

Example

```
const buffer = new ArrayBuffer(4); // 4 bytes
const view = new Uint8Array(buffer);
view[0] = 255;
console.log(view); // Uint8Array [255, 0, 0, 0]
```

Signed vs Unsigned Values

Unsigned Values

- Can only store positive numbers (including 0).
- No space reserved for negatives.
- **Example (8-bit):**
 - Range: 0 to 255
 - Formula: 0 to $(2^n - 1)$

Signed Values

- Can store both positive and negative numbers.

- 1 bit used for the sign (0 = +, 1 = -).
- **Example (8-bit):**
 - Range: -128 to +127
 - Formula: $-(2^{(n-1)})$ to $(2^{(n-1)} - 1)$

Differences

Feature	Signed	Unsigned
Includes Negatives	☑ Yes	✗ No
Range (8-bit)	-128 to 127	0 to 255
Use Case	General math	Bit-level operations

Reading and Writing to ArrayBuffers (Node.js / JS)

ArrayBuffer Basics

- Raw binary data container.
- Use TypedArray or DataView to read/write.

Writing with TypedArray

```
const buffer = new ArrayBuffer(4); // 4 bytes
const view = new Uint8Array(buffer); // 8-bit unsigned view

view[0] = 10;
view[1] = 20;
view[2] = 30;
view[3] = 40;
```

☑ Now buffer holds: [10, 20, 30, 40]

Reading with TypedArray

```
console.log(view[0]); // 10
console.log(view[1]); // 20
```

- You read like reading from normal arrays using typed views.

Using DataView (for more control)

```
const buffer = new ArrayBuffer(4);
const dv = new DataView(buffer);
```

```
dv.setUint8(0, 255);           // write 1 byte ; (unsigned => setUint8, getUint8),
                                (signed => setInt8, getInt8)
dv.setInt16(1, -300);          // write 2 bytes ; (signed)
console.log(dv.getUint8(0));    // 255
console.log(dv.getInt16(1));    // -300
```

- ☑️ DataView helps with:
- Endianness control
 - Signed/Unsigned values
 - Multi-byte values (e.g., 16-bit, 32-bit)

📌 Summary

Method	Use Case
TypedArray	Simple numeric arrays
DataView	Full control over data

📦 Buffer in Node.js

🔧 Creating a Buffer:

```
// import { Buffer } from 'buffer'; // You can also import it.

const buf = Buffer.from("Hello"); // Creates a buffer from a string
```

🔮 Common Methods:

Method	Description
<code>Buffer.from(string)</code>	Create buffer from string
<code>Buffer.alloc(size)</code>	Create buffer of fixed size (filled with 0)
<code>buf.toString()</code>	Convert buffer back to string
<code>buf.length</code>	Get size in bytes
<code>buffer.byteLength</code>	the actual byte length
<code>buf[index]</code>	Access a specific byte
<code>buf.write(string)</code>	Write to buffer

🗎 Example:

```
const buf = Buffer.from('Node.js');
console.log(buf);           // <Buffer 4e 6f 64 65 2e 6a 73>
console.log(buf.toString()); // Node.js
```

Buffer vs String:

- Strings are for **text** (UTF-8).
- Buffers are for **raw binary data**.

Use Cases:

- Reading files in chunks
- Streaming data
- Network operations (TCP/UDP)
- Binary protocol implementations

Buffer.alloc() vs Buffer.allocUnsafe()

Buffer.alloc(size)

- Allocates a **new buffer of the given size**.
- The buffer is **initialized (filled with 0)**.
- **Safe** to use — no old memory data.
- Slightly **slower** because of initialization.

```
const safeBuf = Buffer.alloc(10);
console.log(safeBuf); // <Buffer 00 00 00 00 00 00 00 00 00 00>
```

Buffer.allocUnsafe(size)

- Allocates a **new buffer of the given size**.
- The buffer is **NOT initialized** — may contain **old/random data**.
- **Faster**, but **unsafe** — use only if you're going to overwrite the buffer data immediately.

```
const unsafeBuf = Buffer.allocUnsafe(10);
console.log(unsafeBuf); // <Buffer e2 45 1f ... > (random data)
```

Summary Table

Method	Initializes to 0	Safe to Use	Speed
<code>Buffer.alloc(size)</code>	☒ Yes	☒ Yes	 Slower

Method	Initializes to 0	Safe to Use	Speed
<code>Buffer.allocUnsafe(size)</code>	✗ No	⚠ No	⚡ Faster

💡 Tip:

Always use `Buffer.alloc()` unless you're sure you'll overwrite every byte of the buffer.

📦 Buffer Pool in Node.js

✅ Definition

- A **Buffer Pool** is a pre-allocated chunk of memory.
- Used to manage small Buffer allocations efficiently.

🎯 Purpose

- Avoid frequent memory allocation.
- Improve performance in I/O-heavy operations.

⚙️ How It Works

- Node.js creates a buffer pool (e.g., 8KB).
- Small buffers are sliced from this pool.
- When the pool is full → a new one is created.

🚀 Benefits

- Faster memory allocation
- Reduces GC (Garbage Collection)
- Efficient handling of small buffers

🔍 Example (Internal)

```
const pool = Buffer.allocUnsafe(8192); // Create 8KB buffer pool
const smallBuf = pool.slice(0, 100); // Allocate 100 bytes from pool
```

Note: Buffer pooling is mostly internal in Node.js. You don't usually manage it directly.

🚫 Limitations of Buffer in Node.js

1. 📦 Fixed Size

- Buffer size is set at creation and **cannot be changed** later.

2. 🧠 Manual Memory Handling

- Developers must manage **offsets and lengths** carefully.

3. ✖ Security Risk

- `Buffer.allocUnsafe()` may return **uninitialized memory**.

4. 🐢 Memory Bloat

- Excessive or careless use may cause **high memory usage**.

5. 🗃 Not Type-Safe

- Buffers handle raw binary data, **not suited for complex data types**.

Buffers are powerful but need careful handling to avoid bugs or memory issues.

Base64 Encoding

☒ What is Base64?

- Encodes binary data into ASCII text format.
- Uses 64 characters: A-Z, a-z, 0-9, +, /
- Padding character: = used to complete the block.
- Base64 uses 6 bits to represent a character.

🔗 Why Use It?

- To safely transmit binary data (e.g., images, files) over text-only channels like:
 - HTTP
 - Email
 - JSON

Base64 in Node.js

Encode to Base64

```
const data = "Hello Afnur";
const encoded = Buffer.from(data).toString('base64');
console.log(encoded); // SGVsbG8gQWZudXI=
```

Decode from Base64

```
const decoded = Buffer.from(encoded, 'base64').toString('utf8');  
console.log(decoded); // Hello Afnur
```



Use Cases

- Embedding images in HTML or CSS
- Storing binary in JSON
- File transfer or attachments over email